

四電一丙 數位邏輯實習

第 9 次實習作業報告

組員 1 學號:B812111001 姓名: 張博翔

組員 2 學號: 114310326 黃子軒

組員 1 於此作業中的貢獻:解題、digital、Quartus、協助隊友排錯、作業二、作業報告(二次編排、修改與美編)

組員 2 於此作業中的貢獻:解題、digital、quartus、作業一、作業報告(初步編排、複審內容)

I. 實驗目的

A. 作業 1

用 Mealy 狀態機來設計一個 Sequential Detector 偵測「1001」輸入序列 (自行決定是否加入 4 bits 重製一次的功能)並於報告中詳細說明，請用以下兩種模式的其中一種完成本題的電路設計：

模式一：VHDL 兩種設計模式(Process、Component、布林代數式)；模式二：VHDL 一種設計模式 + 拉電路

B. 作業 2

利用 Mealy 或 Moore 狀態機設計一個自動控制電路，用來自動控制 Lab07 設計的累加減法器(SRAM+加減法器)，請用以下兩種模式的其中一種完成本題的自動控制電路：

模式一：VHDL 兩種設計模式(Process、Component、布林代數式)，模式二：VHDL 一種設計模式 + 拉電路

II. 設計程序

A. 作業 1

1. 用 Mealy 狀態機來設計 Sequential Detector 偵測「1001」輸入序列，不考慮加入重製系統，選用了模式一，設計兩個 VHDL (Process VHDL、布林代數式 VHDL)。

S0 是初始狀態，沒有接收到任何東西，依序接著是 S1 是為已接收到 1，S2 是為已接收到 10，S3 是為已接收到 100，當在 S3 接收到 1 的時候，輸出 Z 為 1，並且回到 S1(因為最後一個 1 可以是下一個序列的開頭的 1)。將狀態圖轉換為狀態表，接著做卡諾圖進行化簡，最後根據卡諾圖化簡結果，最終得到布林代數的邏輯，以下進行推導布林代數式的過程，最後用 VHDL 程式碼撰寫狀態機。

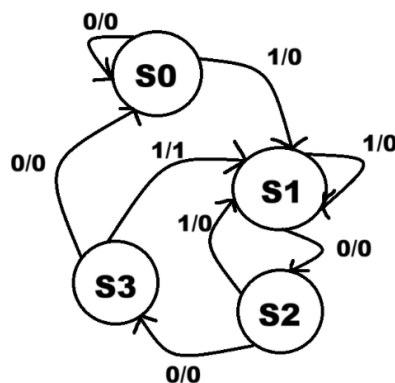


圖 1. Sequential Detector state graph (step1)

表 1. Sequential Detector state table (step2)

AB	A+ B+		Z	
	X=0	X=1	X=0	X=1
00	0 0	0 1	0	0
01	1 0	0 1	0	0
10	1 1	0 1	0	0
11	0 0	0 1	0	1

表 2. Sequential Detector excitation table (step3) (使用 D type Flip Flop)

現態 AB	輸入 X	次態 A+ B+	激勵輸入 $D_1 D_0$	輸出 Z
00	0	00	00	0
00	1	01	01	0
01	0	10	10	0
01	1	01	01	0
10	0	11	11	0
10	1	01	01	0
11	0	00	00	0
11	1	01	01	1

接著將 A+ B+ 值代入卡諾圖進行化簡，求出布林代數式。(step4、5)

AB	X	
	0	1
00	0	0
01	1	0
11	0	0
10	1	0

A+

圖 2. $A+ = X' + B \oplus A$

AB	X	
	0	1
00	0	1
01	0	1
11	0	1
10	1	1

B+

圖 3. $B+ = X + AB'$

AB	X	
	0	1
00	0	0
01	0	0
11	0	1
10	0	0

Z

圖 4. $Z = XAB$

總結布林代數式， $A+ = X'+B\oplus A$ ， $B+ = X+AB'$ ， $Z = XAB$ ，以推導解果編寫 VHDL，並模擬驗證。

2. 以下為 Sequential Detector VHDL 程式碼設計與講解。

Method 1: Boolean Algebra

```
-- =====  
-- 檔案說明：以布林方程式實作的序向邏輯電路 (Mealy Machine)  
-- 使用兩個 D 型正反器 (A、B) 作為狀態暫存器  
-- 輸入：clk (時脈)、X (外部輸入)  
-- 輸出：Z (組合輸出，取決於現態與輸入)  
-- =====  
  
library IEEE;  
use IEEE.STD_LOGIC_1164.ALL;  
  
entity boolean is  
    Port (  
        clk : in  STD_LOGIC; -- 時脈信號，上升緣觸發  
        X   : in  STD_LOGIC; -- 外部輸入  
        Z   : out STD_LOGIC; -- 輸出 (Mealy 輸出，與現態及輸入有關)  
    );  
end boolean;  
  
architecture Equation of boolean is  
  
    -- 狀態暫存器：A 為高位元、B 為低位元，初始狀態為 "00"  
    signal A, B : STD_LOGIC := '0';  
  
    -- D 型正反器激勵輸入 (次態邏輯的組合電路輸出)  
    signal DA, DB : STD_LOGIC;  
  
begin  
  
    -- -----  
    -- 次態邏輯 (D 型正反器激勵方程式)  
    -- DA 為正反器 A 的激勵輸入，決定下一個時脈後 A 的狀態  
    -- 由激勵表化簡得：DA = (not X) and (A xor B)  
    -- -----  
    DA <= (not X) and (A xor B);  
  
    -- DB 為正反器 B 的激勵輸入，決定下一個時脈後 B 的狀態  
    -- 由激勵表化簡得：DB = X or (A and not B)  
    DB <= X or (A and not B);  
  
    -- -----  
    -- 輸出邏輯 (Mealy 輸出方程式)  
    -- Z 僅在現態為 "11" 且輸入 X = 1 時輸出 1  
    -- 由輸出表化簡得：Z = A and B and X  
    -- -----  
    Z <= A and B and X;  
  
    -- -----  
    -- 狀態暫存器更新 (時序電路)  
    -- 於時脈上升緣將激勵輸入存入正反器，完成狀態轉移  
    -- -----  
    process(clk)  
    begin  
        if rising_edge(clk) then  
            A <= DA; -- 更新高位元狀態  
            B <= DB; -- 更新低位元狀態  
        end if;  
    end process;  
  
end Equation;
```

圖 5. Sequential Detector VHDL using Boolean Algebra

VHDL 程式碼講解：

輸入訊號 X、CLK，輸出訊號 Z，偵測時脈的上升邊緣觸發，由兩個 D 正反器加上一些組合邏輯合成，其狀態意義為 00 初始狀態，01 已經有 1，10 是已經有 10，11 是已經到 100，當在輸入 1 時，輸出 Z = 1，狀態回到 01。

以上 VHDL 主要模擬電路中邏輯閘與正反器的關係。

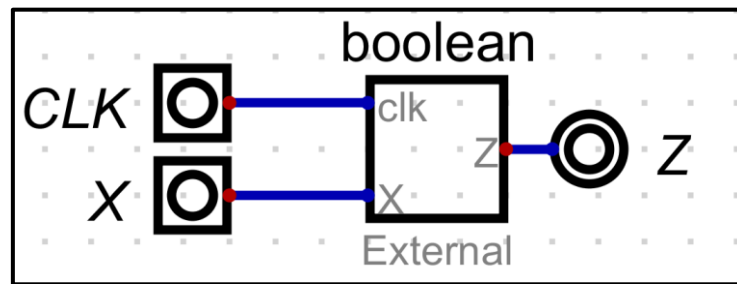


圖 6. Sequential Detector Vhdl 模擬電路 (Boolean Algebra)

Method 2: Process

```
--
-- 檔案說明：序列偵測器 (Sequence Detector)
-- 功能：偵測輸入序列中是否出現 "1001"
-- 同時以 3 位元移位暫存器記錄最近 3 筆輸入
-- 輸入：clk (時脈)、X (串列輸入)
-- 輸出：Z (偵測到 "1001" 時輸出 1)
-- Q3/Q2/Q1 (移位暫存器輸出，顯示最近輸入)
--
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity db12 is
    Port (
        clk : in  STD_LOGIC; -- 時脈信號，上升緣觸發
        X    : in  STD_LOGIC; -- 串列輸入
        Z    : out STD_LOGIC; -- 序列偵測輸出 (偵測到 "1001" 時為 1)
        Q3   : out STD_LOGIC; -- 移位暫存器最新位元 (shift reg[0])
        Q2   : out STD_LOGIC; -- 移位暫存器中間位元 (shift reg[1])
        Q1   : out STD_LOGIC; -- 移位暫存器最舊位元 (shift reg[2])
    );
end db12;

architecture Behavioral of db12 is

    -- 定義 FSM 狀態型別，共 4 個狀態對應序列 "1001" 的接收進度
    -- S0：初始/重置狀態
    -- S1：已接收 "1"
    -- S2：已接收 "10"
    -- S3：已接收 "100"，等待最後一個 '1'
    type state_type is (S0, S1, S2, S3);
    signal state : state_type := S0; -- 狀態暫存器，初始為 S0

    -- 3 位元移位暫存器，用於記錄最近 3 個時脈的輸入值
    -- shift reg(2) 為最舊，shift reg(0) 為最新
    signal shift_reg : std_logic_vector(2 downto 0) := (others => '0');
```

```

begin

process(clk)
begin
    if rising edge(clk) then
        -----
        -- FSM 狀態機：偵測序列 "1001"
        -- 狀態轉移圖：
        -- S0 --1--> S1 --0--> S2 --0--> S3 --1--> S1 (Z=1)
        -- S0 --0--> S0
        -- S1 --1--> S1
        -- S2 --1--> S1
        -- S3 --0--> S0
        -----
        case state is

            when S0 =>
                -- 初始狀態，等待序列的第一個 '1'
                Z <= '0';
                if X = '1' then
                    state <= S1; -- 接收到 '1'，進入 S1
                else
                    state <= S0; -- 接收到 '0'，維持 S0
                end if;

            when S1 =>
                -- 已接收 "1"，等待 '0'
                Z <= '0';
                if X = '0' then
                    state <= S2; -- 接收到 '0'，序列為 "10"，進入 S2
                else
                    state <= S1; -- 接收到 '1'，序列重新從 "1" 開始，維持 S1
                end if;

            when S2 =>
                -- 已接收 "10"，等待第二個 '0'
                Z <= '0';
                if X = '0' then
                    state <= S3; -- 接收到 '0'，序列為 "100"，進入 S3
                else
                    state <= S1; -- 接收到 '1'，序列重新從 "1" 開始，回到 S1
                end if;

            when S3 =>
                -- 已接收 "100"，等待最後一個 '1' 完成序列 "1001"
                if X = '1' then
                    state <= S1; -- 接收到 '1'，序列完整，回到 S1（支援重疊偵測）
                    Z <= '1'; -- 輸出 Z=1，表示偵測到 "1001"
                else
                    state <= S0; -- 接收到 '0'，序列不符，回到初始 S0
                    Z <= '0';
                end if;

            when others =>
                -- 防禦性設計：排除未定義狀態，強制回到 S0
                state <= S0;
                Z <= '0';

        end case;

        -----
        -- 移位暫存器更新：每個時脈將 X 移入最低位元
        -- 新值從右側移入：shift_reg = shift_reg[1:0] & X
        -----
        shift_reg <= shift_reg(1 downto 0) & X;

    end if;
end process;
-----
-- 將移位暫存器各位元輸出至外部接腳
-- Q3 為最新輸入，Q1 為最舊輸入
-----
Q3 <= shift_reg(0); -- 最新輸入（本次時脈）
Q2 <= shift_reg(1); -- 前一個時脈的輸入
Q1 <= shift_reg(2); -- 兩個時脈前的輸入

end Behavioral;

```

圖 7. Sequential Detector using Process

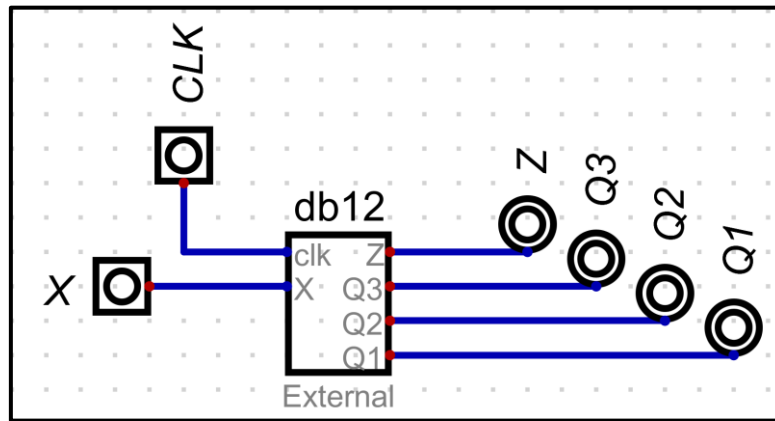


圖 8. Sequential Detector VHDL 模擬電路 (Process)

透過狀態表的行為描述，當有輸入訊號 X、CLK，輸出訊號 Z、Q3、Q2、Q1，偵測輸入序列 1001。

狀態(Q3、Q2、Q1)是為了更好進行判別因此額外拉線。

B. 作業 2

1. 參 lab07 的 SRAM 設計，採用高階資料類別進行設計，以 2 維矩陣描述記憶體架構，程式碼相較描述暫存器接線精簡更多。加法器採用的是 lab03 的加減法器，因為拿掉比較器，因此本次實驗溢位後會扣除 16 繼續累加。

本次目的是將控制線全部整合進行控制，隨著 Clock 進行動作，並且加入 reset 訊號，當 reset = 1 時會將 SRAM 與當前控制狀態清 0 為初始。

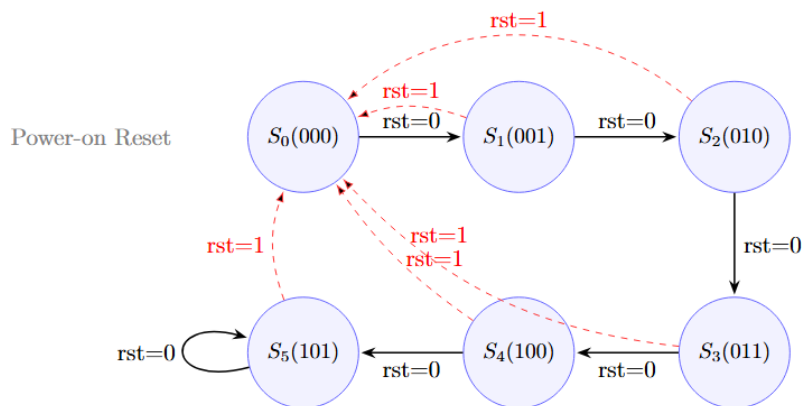


圖 9. SRAM Accumulator state graph (step1)

本次是以 Moore 的方式進行設計，但因為控制線輸出眾多，為求畫面精簡因此未將所有輸出列於狀態下。

表 3. SRAM Accumulator state table (step2)

State	Encoding ($Q_2Q_1Q_0$)	Datapath Control (A, B, C, D)				SRAM Bus (E)			
		sel1	en1	sel2	en2	[Addr	CS	WE	OE]
S_0	000	0	0	0	0	[00	0	0	0]
S_1	001	0	0	0	0	[00	1	1	0]
S_2	010	0	0	0	0	[01	1	1	0]
S_3	011	0	1	0	0	[00	1	0	1]
S_4	100	1	0	1	1	[01	1	0	1]
S_5	101	1	0	1	1	[01	1	1	1]

進行狀態的 encoding，以下為每個狀態對應控制線的動作：

S0：Reset，初始狀態 (控制線全為 0)；

S1：將累加的固定值存入 SRAM 的位置 0 (chip select、write enable、位置指到 0)；

S2：將累加數值存入 SRAM 的位置 1 (chip select、write enable、位置指到 1)；

S3：將固定值存入第一個暫存器，維持定值 (chip select、output enable、位置 0、enable reg1)；

S4：將累加數值存入第二個暫存器，改變 Datapath 經過加減法器

(chip select、output enable、位置 1、切換 select 1 / 2、enable reg2)；

S5：將結果寫回 SRAM 的位置 1 並再次輸出，開始累加

(chip select、write / output enable、位置 1、select 1 / 2、enable reg2)。只要 reset 不 enable 就一直維持 S5 累加。

表 4. SRAM Accumulator excitation table (step3) (使用 D type Flip Flop)

Current State (Q)			Next State Q^+ (rst=0)			Next State Q^+ (rst=1)		
Q_2	Q_1	Q_0	D_2	D_1	D_0	D_2	D_1	D_0
0	0	0	0	0	1	0	0	0
0	0	1	0	1	0	0	0	0
0	1	0	0	1	1	0	0	0
0	1	1	1	0	0	0	0	0
1	0	0	1	0	1	0	0	0
1	0	1	1	0	1	0	0	0

可以看到當 reset 沒有致能時會進入到下個狀態，當 reset 致能時則會清零，回到初始狀態 S0。開始繪製卡諾圖與簡化，求出布林代數式。(step4、5)

(reset 低電位)：

$Q_2 \setminus Q_1 Q_0$	00	01	11	10
0	0	0	1	0
1	1	1	X	X

圖 10. $D_2 = (Q_2 + Q_1 Q_0) \cdot \overline{rst}$

$Q_2 \setminus Q_1 Q_0$	00	01	11	10
0	0	1	0	1
1	0	0	X	X

圖 11. $D_1 = (Q_2' Q_1' Q_0 + Q_1 Q_0') \cdot \overline{rst}$

$Q_2 \setminus Q_1 Q_0$	00	01	11	10
0	1	0	0	1
1	1	1	X	X

圖 12. $D_0 = (Q_2 + Q_0') \cdot \overline{rst}$

$$D_2 = (Q_2 + Q_1 Q_0) \cdot \overline{rst} \quad (1)$$

$$D_1 = (\bar{Q}_2 \bar{Q}_1 Q_0 + Q_1 \bar{Q}_0) \cdot \overline{rst} \quad (2)$$

$$D_0 = (Q_2 + \bar{Q}_0) \cdot \overline{rst} \quad (3)$$

圖 13. D type Flip Flop 輸出布林代數式

依照 State 去決定各控制線的輸出，運用 Boolean Algebra 完成 VHDL 設計。

2. 以下為 Sequential Detector VHDL 程式碼設計與講解。

Method 1: Boolean Algebra

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity CtrSRAM_2 is
    port (
        clk : in std_logic;
        rst : in std_logic;
        sel1 : out std_logic;
        en1 : out std_logic;
        sel2 : out std_logic;
        en2 : out std_logic;
        Addr : out std_logic_vector(1 downto 0);
        CS : out std_logic;
        WE : out std_logic;
        OE : out std_logic
    );
end CtrSRAM_2;

architecture Hybrid of CtrSRAM_2 is
    signal Q : std_logic_vector(2 downto 0) := "000";
    signal D : std_logic_vector(2 downto 0);

    -- 宣告一個 9-bit 的信號，用來當作所有輸出控制線的「總表」
    -- 排列順序：sel1(8), en1(7), sel2(6), en2(5), Addr(4 downto 3), CS(2), WE(1), OE(0)
    signal Ctrl Bus : std_logic_vector(8 downto 0);

begin
    -- =====
    -- 1. 次態邏輯 (嚴格使用 D 的布林代數式)
    -- =====
    D(2) <= Q(2) or (Q(1) and Q(0));
    D(1) <= (not Q(2) and not Q(1) and Q(0)) or (Q(1) and not Q(0));
    D(0) <= Q(2) or not Q(0);

    -- =====
    -- 2. 狀態暫存器 (D 正反器)
    -- =====
    process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                Q <= "000";
            else
                Q <= D;
            end if;
        end if;
    end process;

    -- =====
    -- 3. 輸出邏輯 (直接寫成一張真值表 Lookup Table)
    -- =====
    -- 利用 with ... select 語法，根據當前狀態 Q，直接賦值給 Ctrl Bus
    -- 排列順序：sel1 & en1 & sel2 & en2 & Addr(2 bits) & CS & WE & OE
    with Q select
        Ctrl Bus <=
            -- sel1, en1, sel2, en2, Addr1, Addr0, CS, WE, OE
            "000000000" when "000", -- S0
            "000000110" when "001", -- S1
            "000001110" when "010", -- S2
            "010000101" when "011", -- S3
            "101101101" when "100", -- S4
            "101101111" when "101", -- S5
            "000000000" when others; -- 未使用狀態 S6, S7 預設輸出 0

    -- =====
    -- 4. 將總表的數值解開，接給實際的輸出腳位
    -- =====
    sel1 <= Ctrl Bus(8);
    en1 <= Ctrl Bus(7);
    sel2 <= Ctrl Bus(6);
    en2 <= Ctrl Bus(5);
    Addr <= Ctrl Bus(4 downto 3);
    CS <= Ctrl Bus(2);
    WE <= Ctrl Bus(1);
    OE <= Ctrl Bus(0);

end Hybrid;

```

圖 14. SRAM Accumulator VHDL using Boolean Algebra

VHDL 程式碼講解：

概念為將輸出變成 ROM，因為控制線有 9 條，推導 9 個方程式除了費時也不易維護，因此把 9 條線合成為 Ctrl_Bus，利用 with Q select 在 VHDL 裡面直接寫出一張「表」，方便對照 SRAM Accumulator state table。

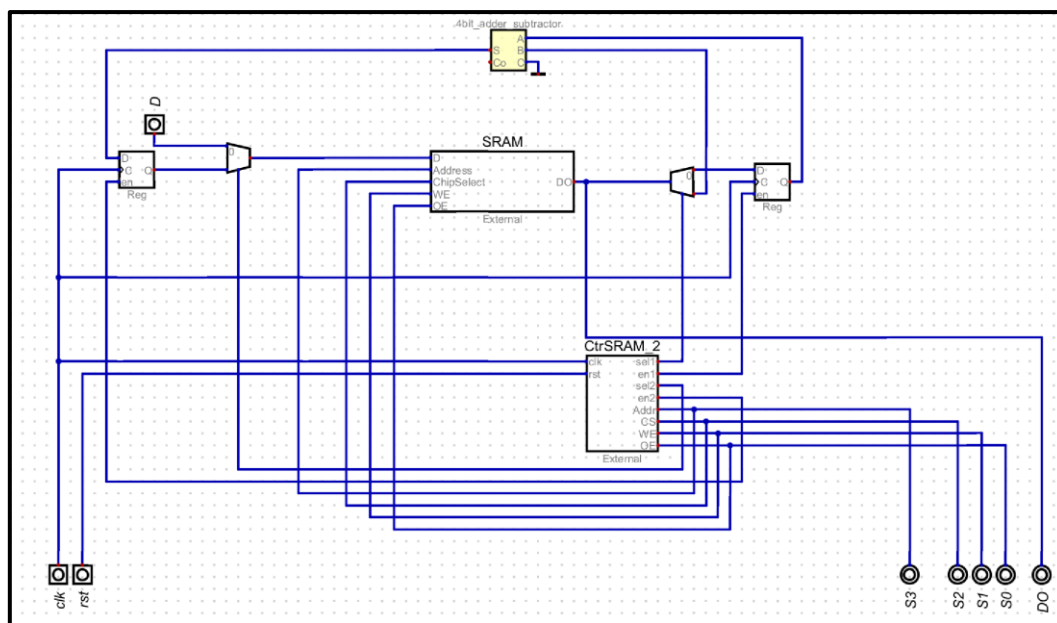


圖 15. SRAM Accumulator VHDL 模擬電路 (Boolean Algebra)

Method 2: Process

```

LIBRARY ieee;
USE ieee.std_logic_1164.all;

entity CtrSRAM is
    port (
        clk : in std_logic;
        rst : in std_logic;
        sel1 : out std_logic;
        en1 : out std_logic;
        sel2 : out std_logic;
        en2 : out std_logic;
        Addr : out std_logic_vector(1 downto 0);
        CS : out std_logic;
        WE : out std_logic;
        OE : out std_logic
    );
end CtrSRAM;

architecture CtrSRAM_arch of CtrSRAM is
    -- 定義狀態機的所有狀態 (使用列舉型別，讓合成器自動進行狀態編碼)
    type state_type is (S0, S1, S2, S3, S4, S5);
    signal current_state, next_state : state_type;
begin
    -- =====
    -- 1. 循序邏輯：狀態暫存器 (State Register)
    -- 功能：負責在時脈 (clk) 的上升緣切換狀態，並處理同步重置 (Reset)。
    -- =====
    process(clk)
    begin
        if rising_edge(clk) then
            if rst = '1' then
                current_state <= S0; -- 當 rst 為 1 時，強制同步回到初始狀態 S0
            else
                current_state <= next_state; -- 否則，將下一個狀態載入成為當前狀態
            end if;
        end if;
    end process;

    -- =====
    -- 2. 組合邏輯：次態邏輯 (Next State Logic)
    -- 功能：根據「當前狀態 (current_state)」決定「下一個狀態 (next_state)」。
    -- 注意：這是一個純組合邏輯，只要 current_state 或 rst 改變，就會立刻觸發重新計算。
    -- =====
    process(current_state, rst)
    begin
        case current_state is
            when S0 =>
                -- 特別處理 S0 狀態：確保在 rst 放開前，次態不會偷跑
                if rst = '1' then
                    next_state <= S0; -- rst 期間，次態鎖定在 S0
                else
                    next_state <= S1; -- rst 放開 (變為 0) 後，次態才指向 S1
                end if;
            when S1 => next_state <= S2; -- 線性狀態轉移
            when S2 => next_state <= S3;
            when S3 => next_state <= S4;
            when S4 => next_state <= S5;
            when S5 => next_state <= S5; -- 狀態 S5 為終點，自我迴圈鎖死在此狀態
            when others => next_state <= S0; -- 防呆機制：若發生預期外的狀態，自動回到 S0
        end case;
    end process;
end CtrSRAM_arch;

```

```

-- 3. 組合邏輯：輸出邏輯 (Output Logic - Moore Machine)
-- 功能：僅根據「當前狀態 (current state)」來決定所有控制線的輸出。
process(current state)
begin
    -- 【重要設計技巧】預設輸出值 (Default Values)
    -- 在進入 case 判斷前，先將所有輸出設為 0 (或預設安全值)。
    -- 這樣可以避免產生非預期的鎖存器 (Latch)，也讓後面的 case 區塊更簡潔。
    sel1 <= '0'; en1 <= '0'; sel2 <= '0'; en2 <= '0';
    Addr <= "00"; CS <= '0'; WE <= '0'; OE <= '0';

    -- 根據當前狀態，覆寫需要變為 '1' 或特定數值的控制線
    case current state is
        when S0 =>
            -- S0 為閒置/重置狀態，所有控制線維持預設值 0
            sel1 <= '0'; en1 <= '0'; sel2 <= '0'; en2 <= '0';
            Addr <= "00"; CS <= '0'; WE <= '0'; OE <= '0';
        when S1 =>
            -- 進入 S1：啟動 SRAM 寫入準備 (Address = 00)
            sel1 <= '0'; en1 <= '0'; sel2 <= '0'; en2 <= '0';
            Addr <= "00"; CS <= '1'; WE <= '1'; OE <= '0';
        when S2 =>
            -- 進入 S2：繼續 SRAM 寫入 (Address = 01)
            sel1 <= '0'; en1 <= '0'; sel2 <= '0'; en2 <= '0';
            Addr <= "01"; CS <= '1'; WE <= '1'; OE <= '0';
        when S3 =>
            -- 進入 S3：開啟 SRAM 讀取 (OE=1)，並致能 Datapath 的 en1
            sel1 <= '0'; en1 <= '1'; sel2 <= '0'; en2 <= '0';
            Addr <= "00"; CS <= '1'; WE <= '0'; OE <= '1';
        when S4 =>
            -- 進入 S4：設定 Datapath 選擇線與致能，讀取 SRAM Address 01
            sel1 <= '1'; en1 <= '0'; sel2 <= '1'; en2 <= '1';
            Addr <= "01"; CS <= '1'; WE <= '0'; OE <= '1';
        when S5 =>
            -- 進入 S5：同時開啟 SRAM 的讀寫 (WE=1, OE=1)，維持 Datapath 設定
            sel1 <= '1'; en1 <= '0'; sel2 <= '1'; en2 <= '1';
            Addr <= "01"; CS <= '1'; WE <= '1'; OE <= '1';
        when others =>
            null; -- 若發生未定義狀態，不做任何事 (會套用最上方的預設值 0)
    end case;
end process;
end CtrSRAM arch;

```

圖 16. SRAM Accumulator using Process

VHDL 程式碼講解：

與前一種依賴卡諾圖推導、手動寫出每條布林代數式的 **Dataflow** 寫法相比，這種狀態機的寫法將設計思維提升到了 **Behavioral**。

前者雖然能精確展現底層的邏輯閘連線，但只要規格稍有變動，就必須重新畫表計算整套方程式，維護成本極高且容易出錯；而後者則是直接將狀態轉移圖與真值表映射為直觀的 **case** 語法，把繁雜的邏輯化簡工作交由合成軟體自動最佳化，大幅提升了程式碼的可讀性與修改彈性。

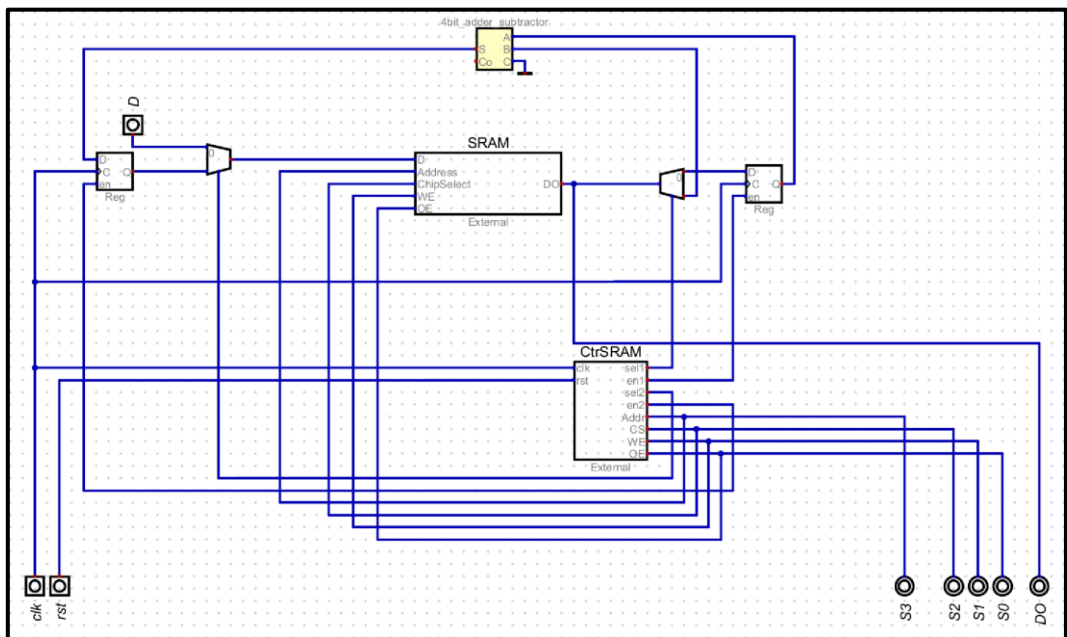


圖 17. SRAM Accumulator VHDL 模擬電路 (Process)

III. 模擬驗證

A. 作業 1

Boolean Algebra VHDL 測試並確認其功能：偵測輸入序列 1001 進行，從 0 為初始點，開始接著輸入 1001，在每一個時脈上升邊緣觸發後，最後輸出 Z 輸出 1。

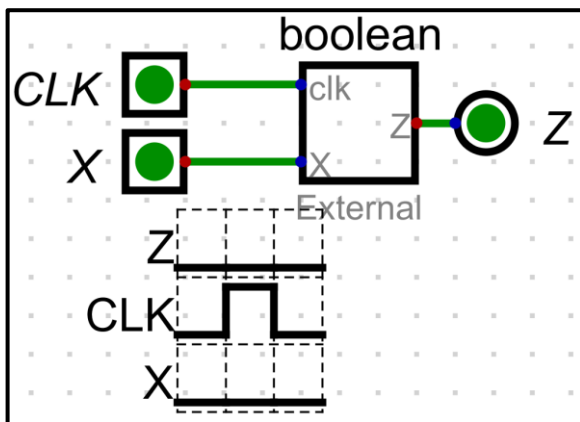


圖 18. Input X = 0; Detect 0; Output Z = 0

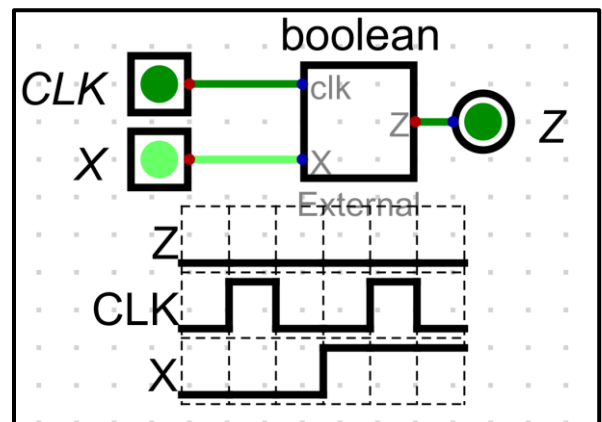


圖 19. Input X = 1; Detect 01; Output Z = 0

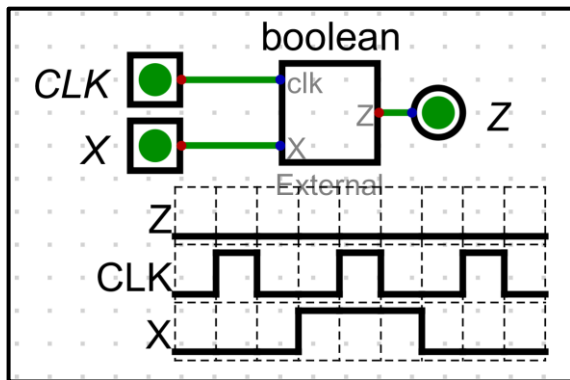


圖 20. Input X = 0; Detect 010; Output Z = 0

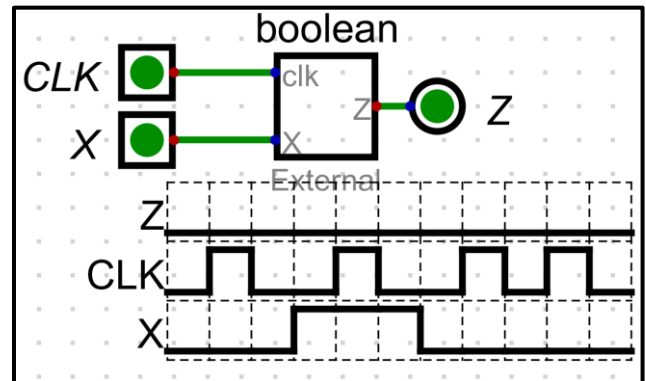


圖 21. Input X = 0; Detect 0100; Output Z = 0

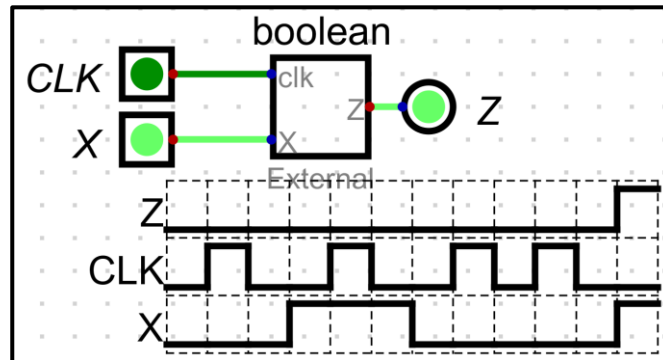


圖 22. Input X = 1; Detect 01001; Output Z = 1

Process VHDL 測試並確認其功能：與第一種不同的是有新增三個輸出用來記錄前三次輸入。

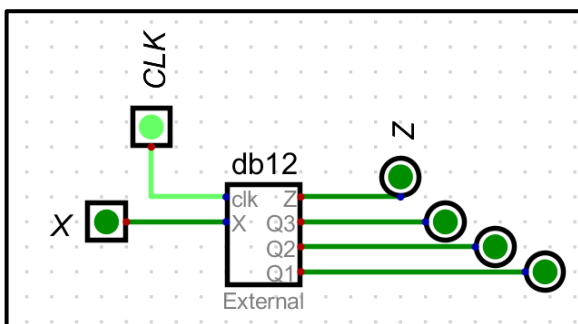


圖 23. Input X = 0; Detect 0; Output Z = 0

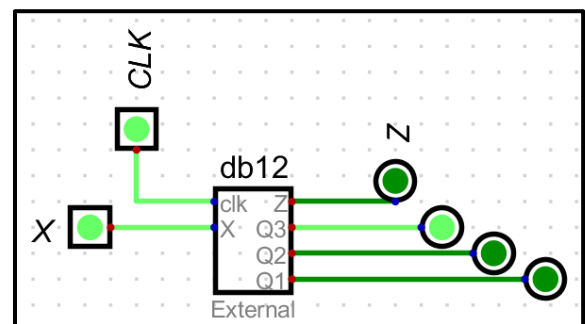


圖 24. Input X = 1; Detect 01; Output Z = 0

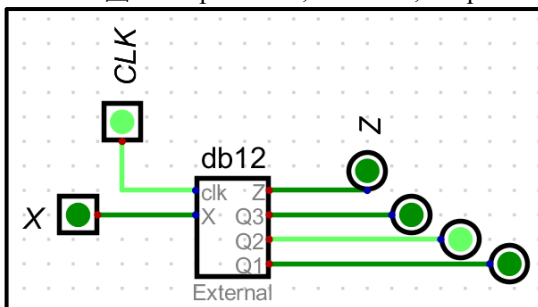


圖 25. Input X = 0; Detect 010; Output Z = 0

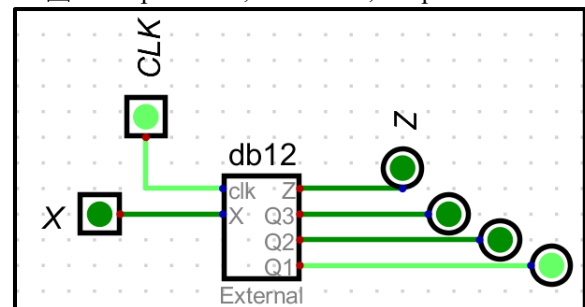


圖 26. Input X = 0; Detect 0100; Output Z = 0

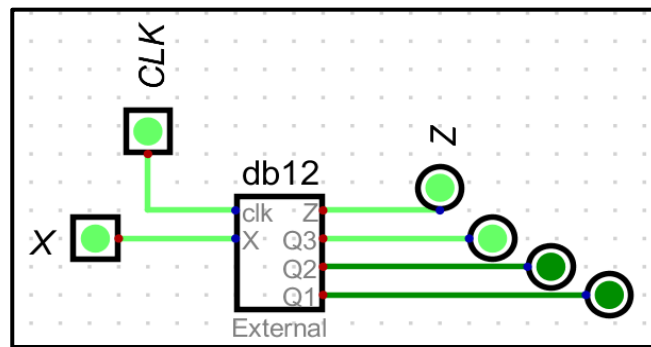


圖 27. Input X = 1; Detect 01001; Output Z = 1

B. 作業 2

SRAM 控制電路測試：10 進位表示，Calculate：起始值(4)+累加值(4)

CLK：正緣觸發

控制線分為 Datapath control 與 SRAM Bus，為了方便判斷提出 SRAM Bus 進行狀態判讀。(燒錄時腳位不夠因此只使用其中一種)

Process VHDL 測試並確認其功能：

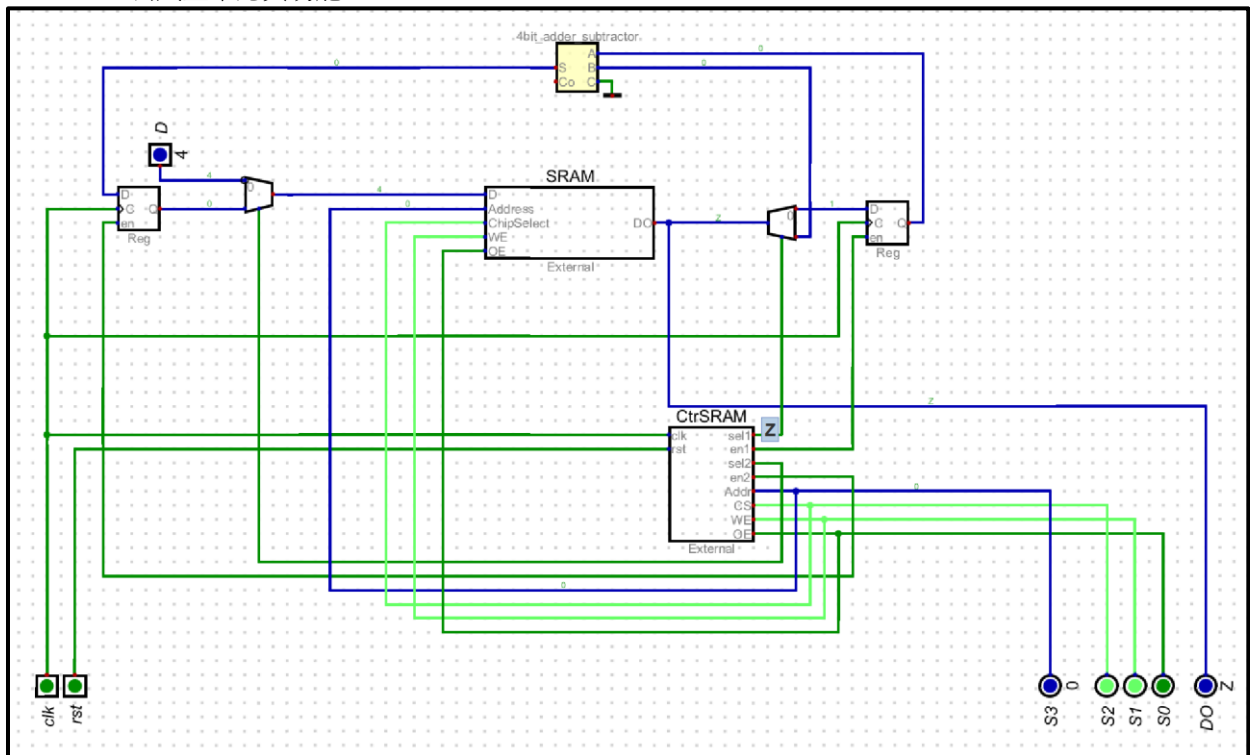


圖 28. Addr0 存入累加值(4) ; Datapath control 0110 (S1) ; SRAM output Z(High impedance) / Accumulator = 0

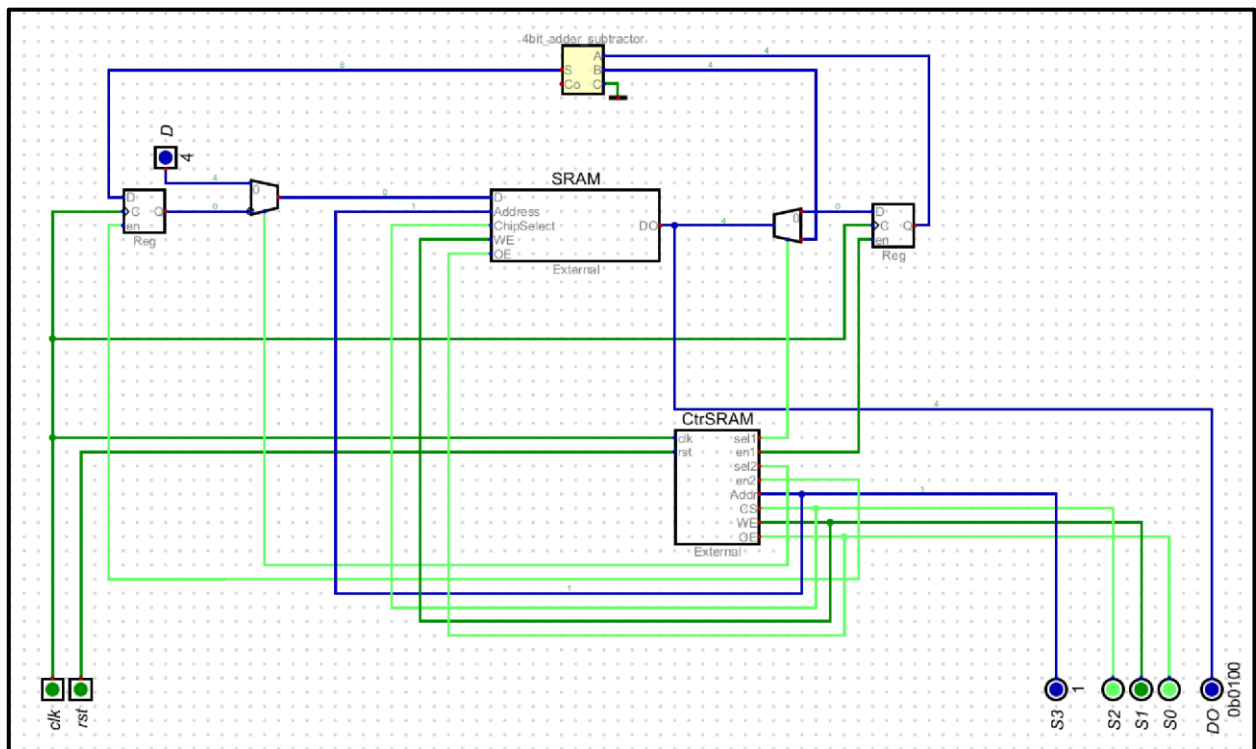


圖 31. 初值(4)經過加法器(+4)後寫入 Register2(8) ; Datapath control 1101 (S4) ; SRAM output 4 / Accumulator = 8

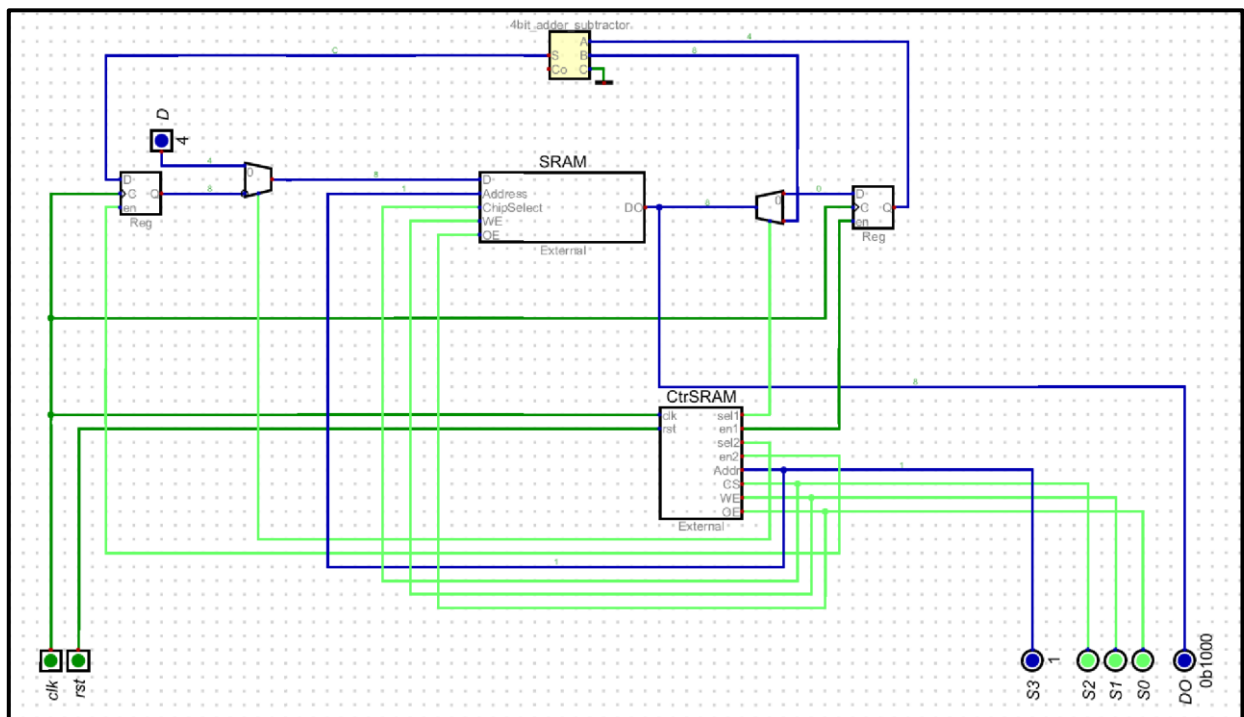


圖 32. 計算結果 Register2(8)寫回 SRAM 再輸出 ; Datapath control 1111 (S5) ; SRAM output 8 / Accumulator = 12(C)

重複 S5 開始累加

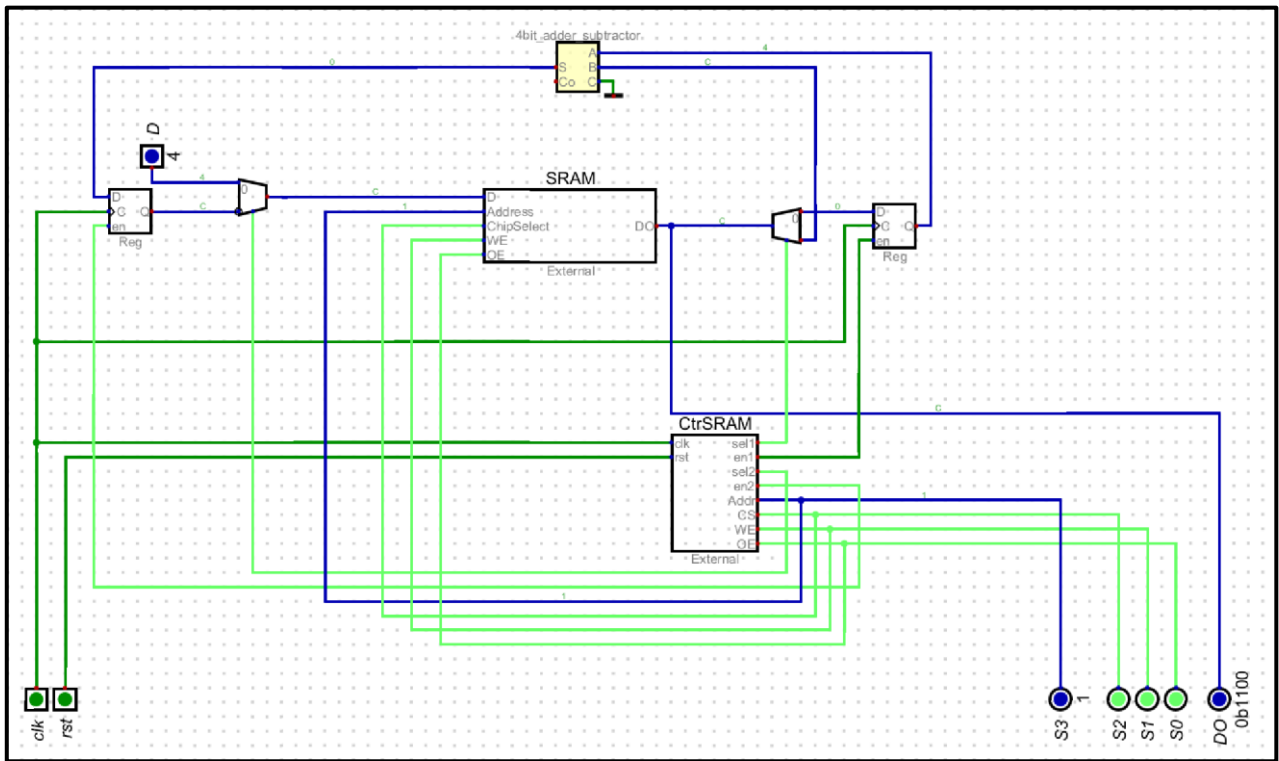


圖 33. Register2(C)寫回 SRAM 再輸出 ; Datapath control 1111 (S5) ; SRAM output 12(C) / Accumulator = 0(OF)

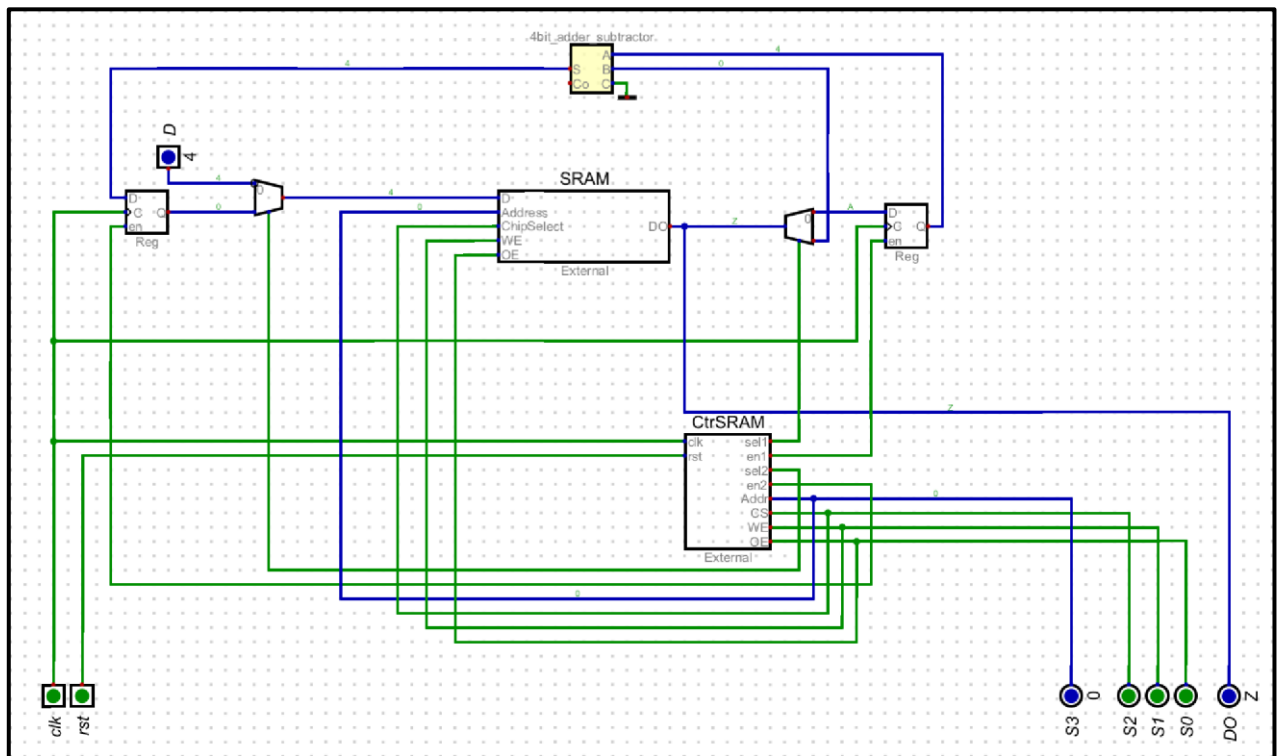


圖 34. RESET 重置 SRAM ; Datapath control 0000 (S0) ; SRAM output Z / Accumulator = 4(先前存在 reg1 未覆蓋)

Boolean Algebra VHDL 測試並確認其功能：

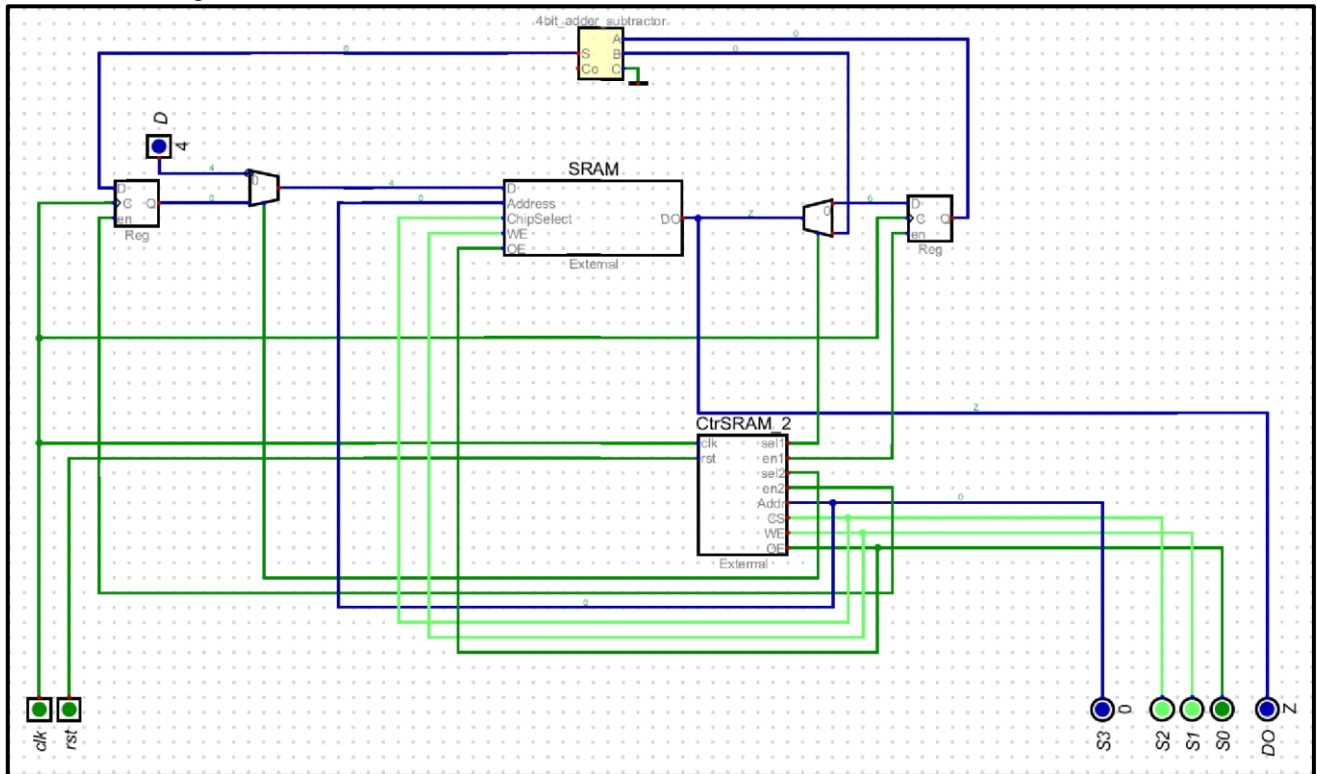


圖 35. Addr0 存入累加值(4) ; Datapath control 0110 (S1) ; SRAM output Z(High impedance) / Accumulator = 0

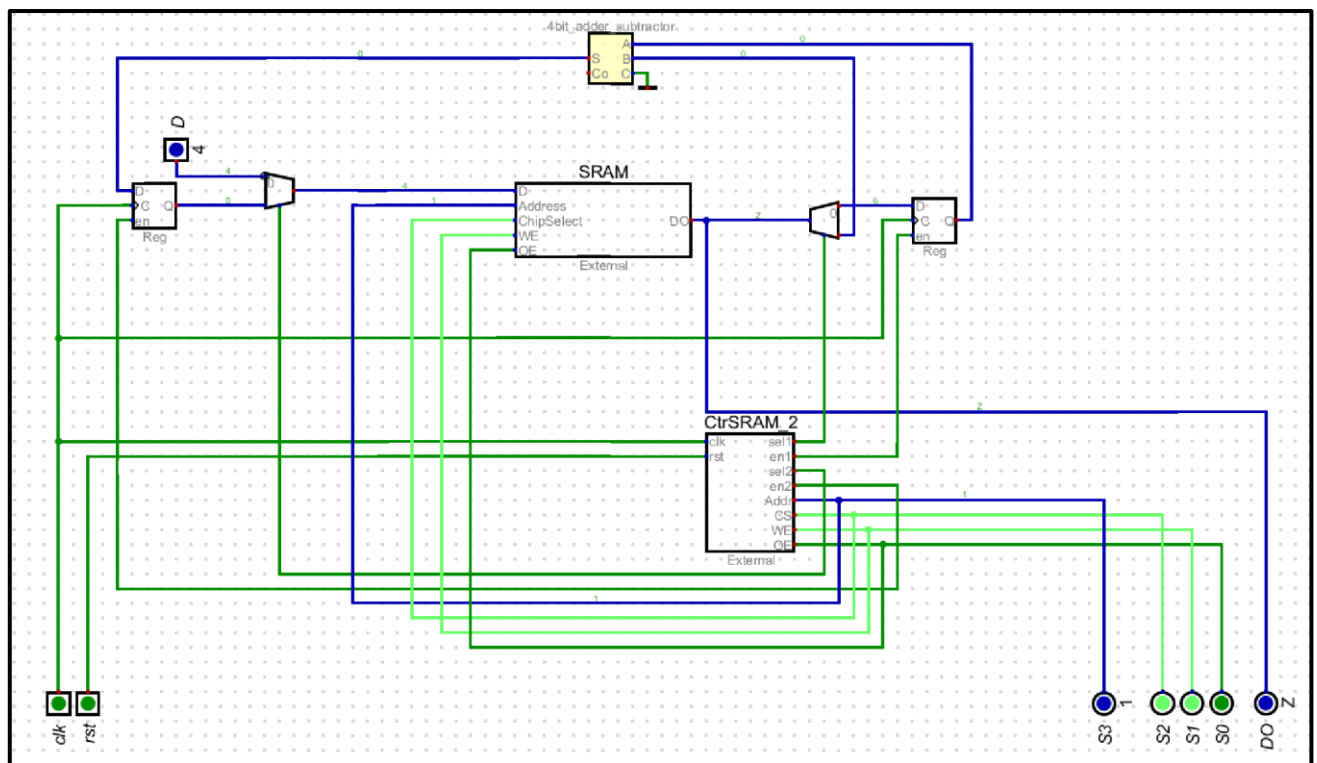


圖 36. Addr1 存入初值(4) ; Datapath control 1110 (S2) ; SRAM output Z(High impedance) / Accumulator = 0

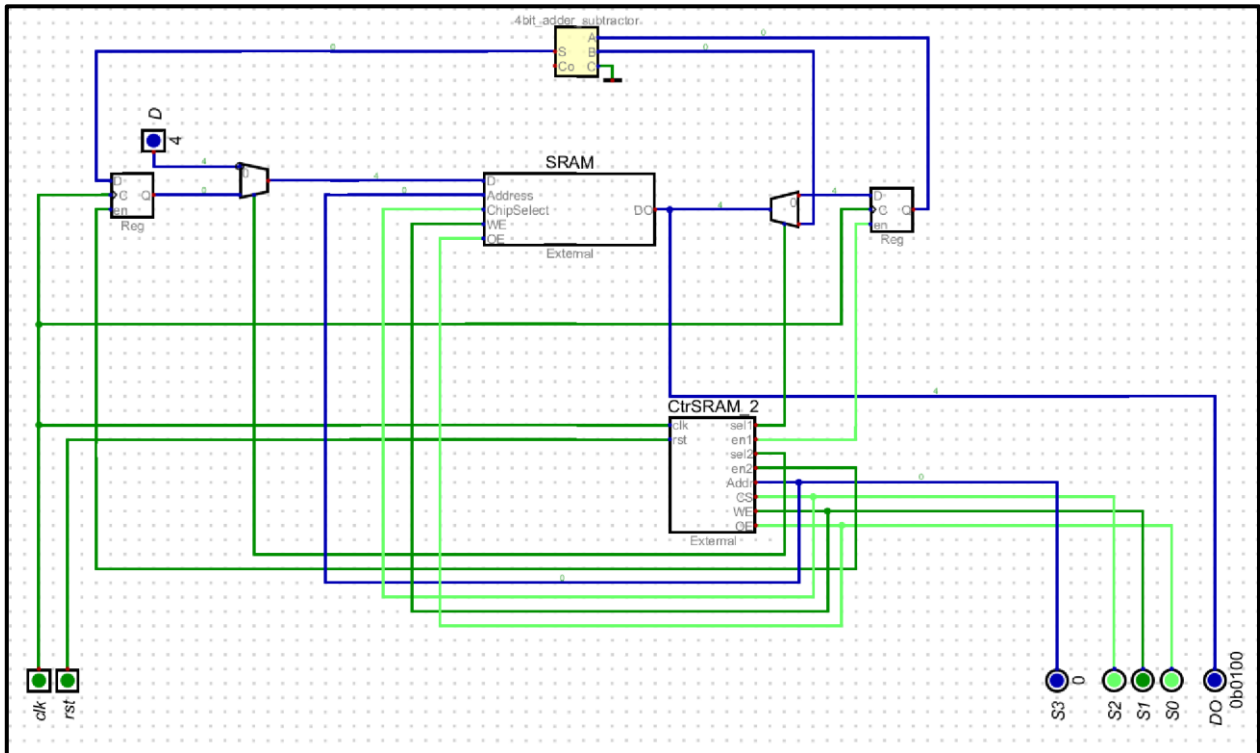


圖 37. 累加值(5)寫入 Register1(4) ; Datapath control 0101 (S3) ; SRAM output 4 / Accumulator = 0(下個 CLK reg1 輸出)

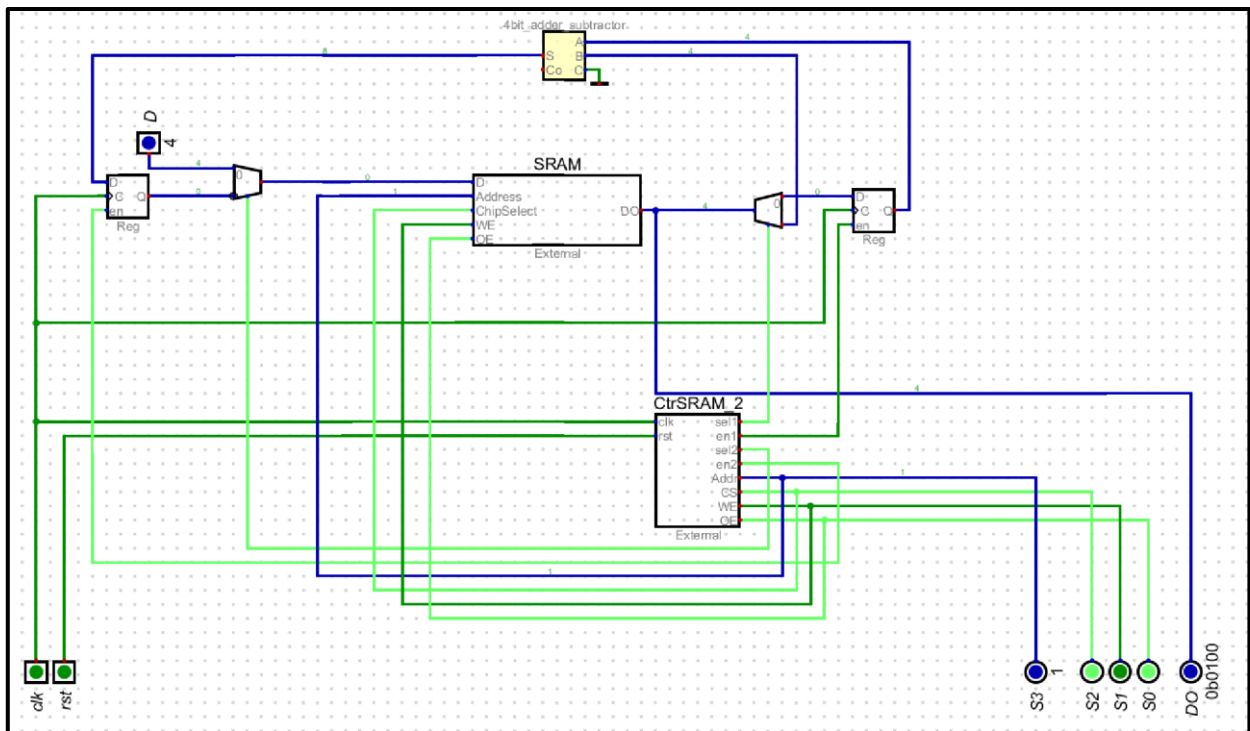


圖 38. 初值(4)經過加法器(+4)後寫入 Register2(8) ; Datapath control 1101 (S4) ; SRAM output 4 / Accumulator = 8

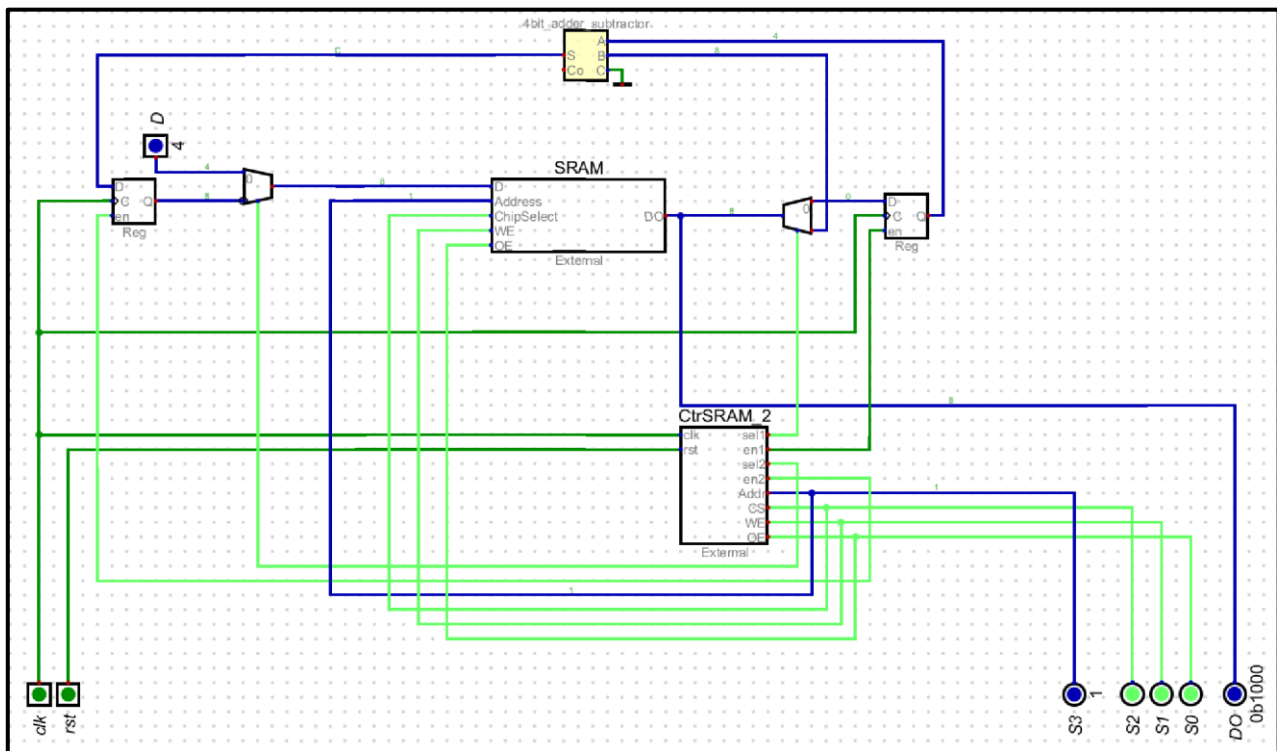


圖 39. 計算結果 Register2(8)寫回 SRAM 再輸出 ; Datapath control 1111 (S5) ; SRAM output 8 / Accumulator = 12(C)

重複 S5 開始累加

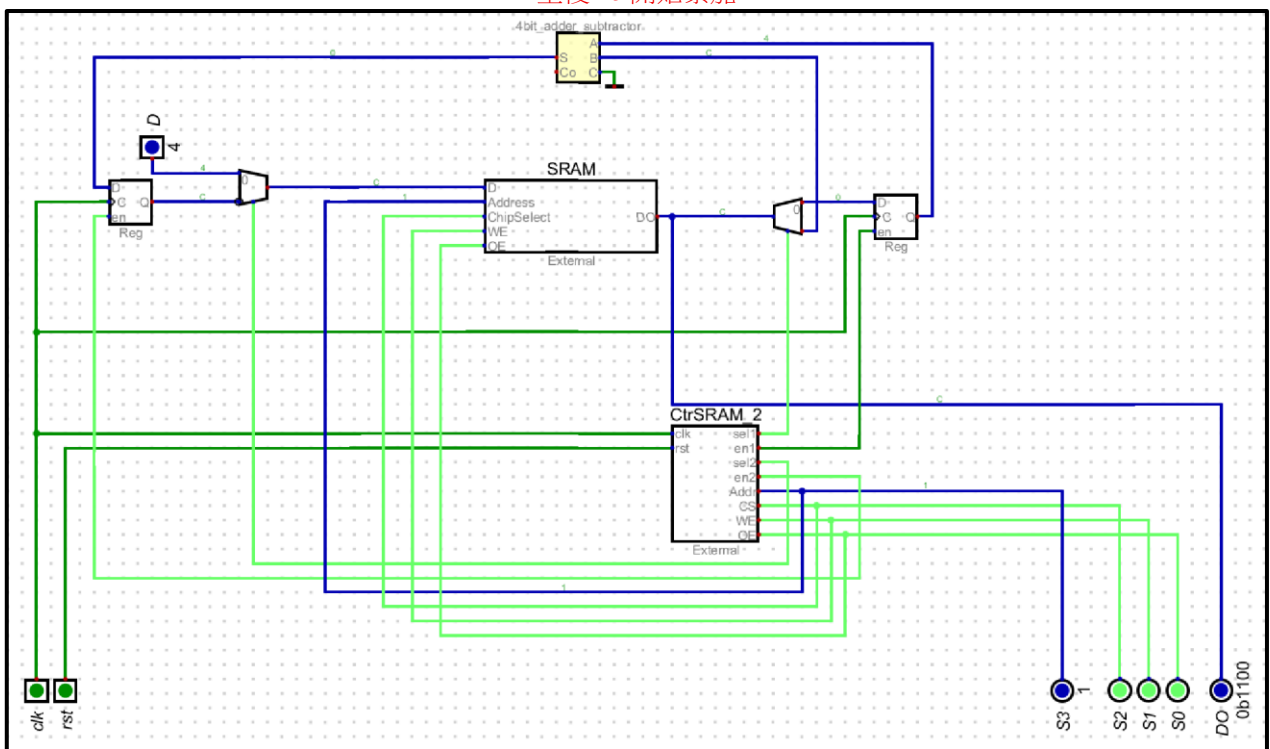


圖 40. Register2(C)寫回 SRAM 再輸出 ; Datapath control 1111 (S5) ; SRAM output 12(C) / Accumulator = 0(OF)

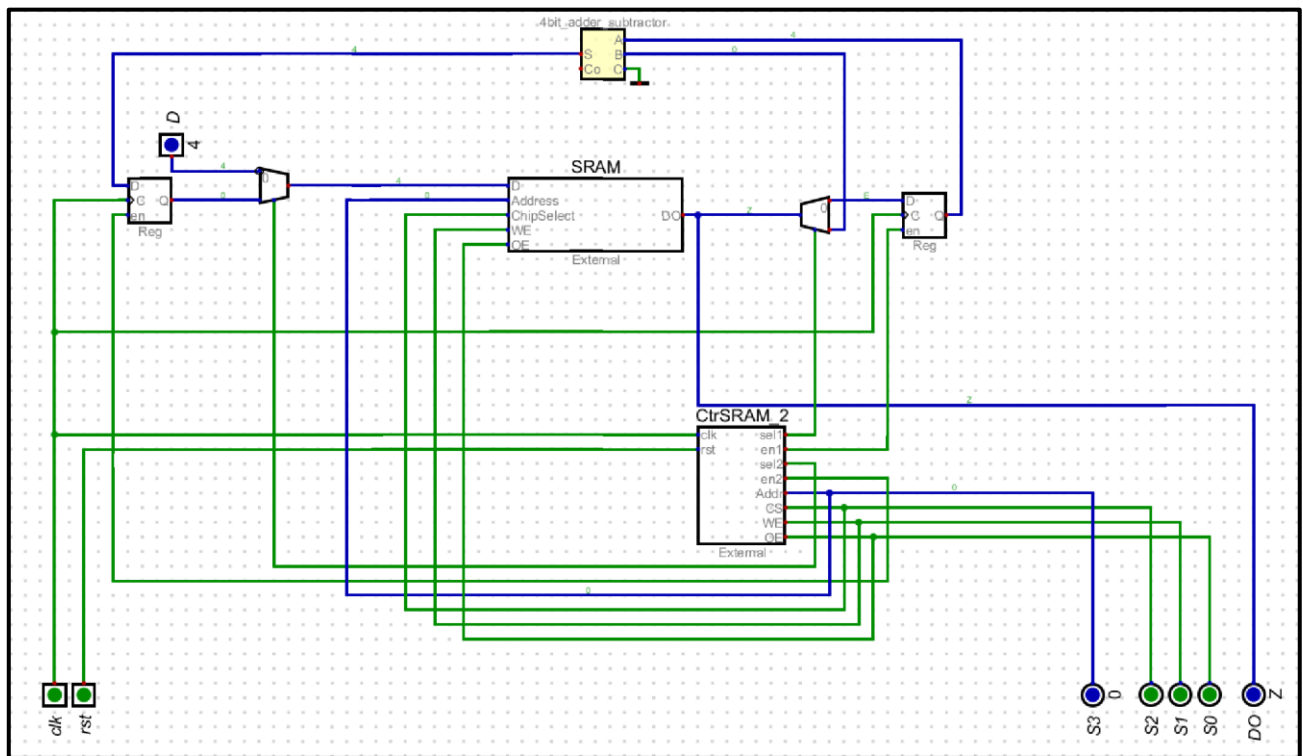


圖 41. RESET 重置 SRAM ; Datapath control 0000 (S0) ; SRAM output Z / Accumulator = 4(先前存在 reg1 未覆蓋)

IV. 實驗結果與成果討論

A. 作業 1

Mealy 狀態機計數器：Input X、CLK；Output Q3、Q2、Q1、Z；Output Q3、Q2、Q1、Z 為 LED3、LED2、LED1、LED0，以下為實驗結果。



圖 42. Input X = 1; Detect 1; Output Z = 0



圖 43. Input X = 0; Detect 10; Output Z = 0

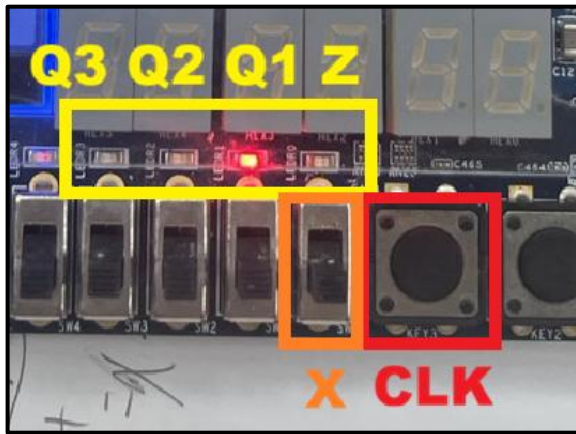


圖 44. Input X = 0; Detect 100; Output Z = 0

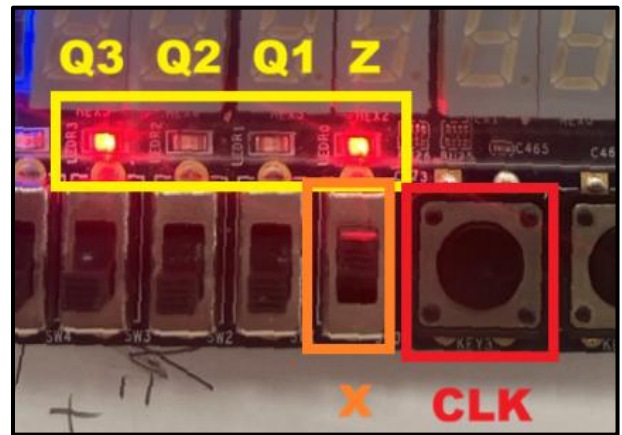


圖 45. Input X = 1; Detect 1001; Output Z = 1

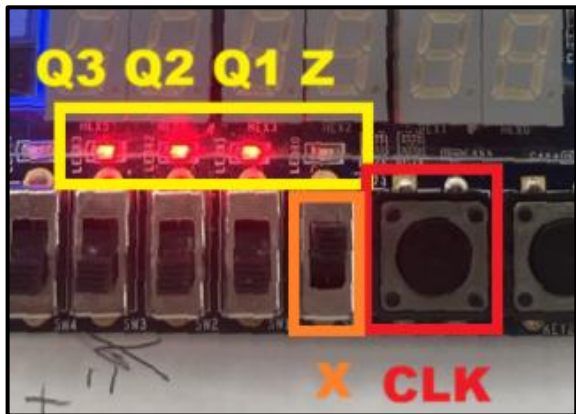


圖 46. Input X = 1; Detect 111; Output Z = 0



圖 47. Input X = 0; Detect 110; Output Z = 0

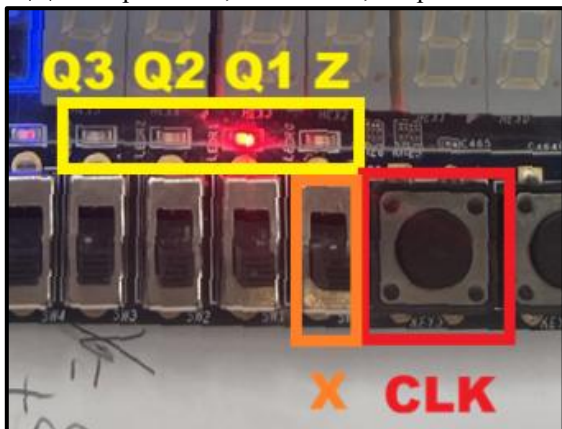


圖 48. Input X = 0; Detect 100; Output Z = 0

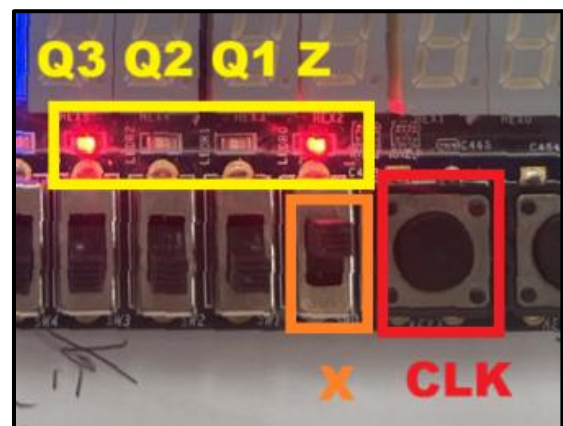


圖 49. Input X = 1; Detect 1001; Output Z = 1

B. 作業 2

Input : reset (Push button 2) 、 clk (Push button 3) 、 Data (Switch 3~0) ;

Output : SRAM bus / State (LED 8~4) 、 Data Output (LED 3~0) , 以下為實驗結果。

[Address 是 2 bits , 但因為只用到 0、1 , 因此 LED8 恆為 0 , 從 LED7 開始判斷]



圖 50. Addr0 存入累加值(4) ; SRAM bus 0110 (S1) ; SRAM output Z(High impedance) / Accumulator = 0

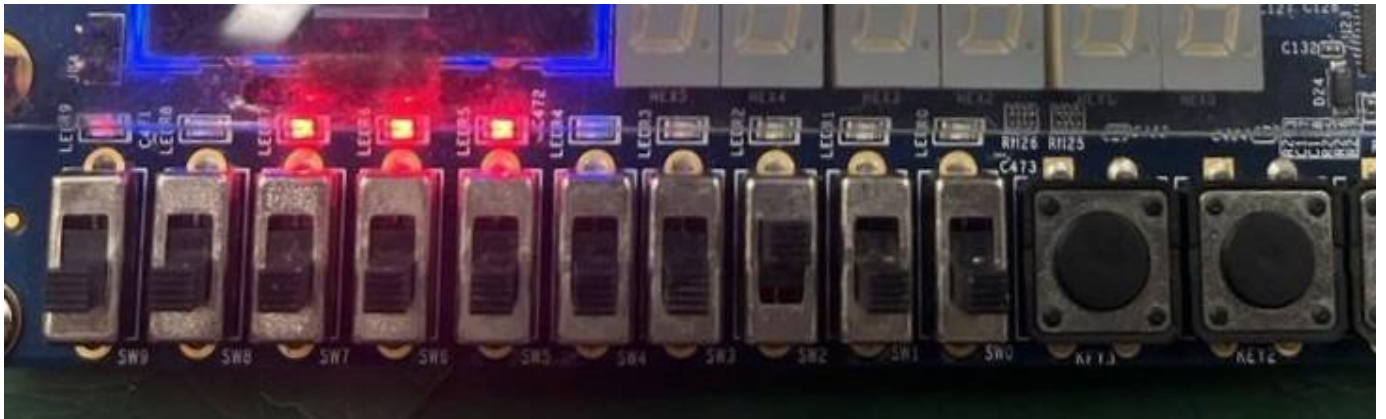


圖 51. Addr1 存入初值(4) ; SRAM bus 1110 (S2) ; SRAM output Z(High impedance) / Accumulator = 0



圖 52. 累加值寫入 Register1 ; SRAM bus 0101 (S3) ; SRAM output 4 / Accumulator = 0(下個 CLK reg1 輸出)



圖 53. 初值(4)經過加法器(+4)後寫入 Register2(8) ; SRAM bus 1101 (S4) ; SRAM output 4 / Accumulator = 8



圖 54. 計算結果 Register2(8)寫回 SRAM 再輸出 ; SRAM bus 1111 (S5) ; SRAM output 8 / Accumulator = 12(C)

重複 S5 開始累加



圖 55. Register2(C)寫回 SRAM 再輸出 ; SRAM bus 1111 (S5) ; SRAM output 12(C) / Accumulator = 0(OF)



圖 56. RESET 重置 SRAM ; SRAM bus 0000 (S0) ; SRAM output Z / Accumulator = 4(先前存在 reg1 未覆蓋)

V. 實驗心得

組員 1：開始與生活中的計算機元件連結，感覺非常有趣且更加容易做連結，希望可以有更多相關主題。

組員 2：這次作業一跟著老師的投影片一步步做還可以，但是當看到組員做作業二才發現作業案二的難度對我來說有點複雜，希望下次的作業可以降一點難度，也要謝謝組員的幫忙。

參考文獻

- [1] DLE 10 數位邏輯實習 MOORE、MEALY 狀態機設計